

**EXP 2 : Construct B-Tree an order of 5 with a set of random elements stored in array.
Implement searching, insertion and deletion operations.**

AIM:

To implement a B-Tree of order 5 in C and perform the basic operations:

- Create a B-Tree
- Insert keys into the B-Tree
- Display the B-Tree (level order and inorder traversal)
- Search for a key in the B-Tree
- Delete a key from a leaf node

PROCEDURE:

1. Define the B-Tree node structure with keys, child pointers, number of keys, and leaf information.
2. Create an empty B-Tree node as the root.
3. Insert the given keys into the B-Tree. Split a node whenever it becomes full.
4. Display the B-Tree using level-order traversal and inorder traversal.
5. Enter a key to delete and remove it if it is present in a leaf node.
6. Display the B-Tree after deletion.
7. Enter a key to search and check whether it is present in the B-Tree.
8. Display the search result as found or not found.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#define ORDER 5

struct BTreeNode
{
    int keys[ORDER-1];
    struct BTreeNode *child[ORDER];
    int n;
    int leaf;
};

//CREATE NODE OPERATION
struct BTreeNode *createNode(int leaf)
{
    struct BTreeNode * newNode = (struct BTreeNode *)malloc(sizeof(struct BTreeNode));
    newNode->leaf = leaf;
    newNode->n = 0;
    int i;
    for(i = 0; i < ORDER; i++)
        newNode->child[i] = NULL;
    return newNode;
}
```

```

}

//SPLIT NODE OPERATION
void splitChild(struct BTreeNode *parent, int index, struct BTreeNode *child)
{
    struct BTreeNode *newNode;
    int mid = (ORDER - 1) / 2;
    newNode = createNode(child->leaf);
    newNode->n = child->n - mid - 1;
    for(int i = 0; i < newNode->n; i++)
    {
        newNode->keys[i] = child->keys[mid + 1 + i];
    }
    if(child->leaf == 0)
    {
        for(int i = 0; i <= newNode->n; i++)
        {
            newNode->child[i] = child->child[mid + 1 + i];
        }
    }
    child->n = mid;
    for(int i = parent->n; i >= index + 1; i--)
    {
        parent->child[i + 1] = parent->child[i];
    }
    parent->child[index + 1] = newNode;
    for(int i = parent->n - 1; i >= index; i--)
    {
        parent->keys[i + 1] = parent->keys[i];
    }
    parent->keys[index] = child->keys[mid];
    parent->n++;
}

// INSERT NON FULL OPERATION
void insertNonFull(struct BTreeNode *node, int key)
{
    int i = node->n - 1;
    if(node->leaf)
    {
        // Shift larger keys
        while(i >= 0 && key < node->keys[i])
        {
            node->keys[i + 1] = node->keys[i];
            i--;
        }
        // Insert key
    }
}

```

```

        node->keys[i + 1] = key;
        node->n++;
    }
    else
    {
        // Find child
        while(i >= 0 && key < node->keys[i])
            i--;
        i++;
        // If child is full
        if(node->child[i]->n == ORDER - 1)
        {
            splitChild(node, i, node->child[i]);
            if(key > node->keys[i])
                i++;
        }
        insertNonFull(node->child[i], key);
    }
}

//INSERT OPERATION
void insert(struct BTreeNode **root, int key)
{
    struct BTreeNode *r = *root;
    // Root is full
    if(r->n == ORDER - 1)
    {
        struct BTreeNode *newRoot = createNode(0);
        *root = newRoot;
        newRoot->child[0] = r;
        splitChild(newRoot, 0, r);
        insertNonFull(newRoot, key);
    }
    else
    {
        insertNonFull(r, key);
    }
}

```

//DISPLAY OPERATION

```

void display(struct BTreeNode *root)
{
    int i;
    if(root == NULL)
        return;

```

```

    for(i = 0; i < root->n; i++)
    {
        if(root->leaf == 0)
            display(root->child[i]);

        printf("%d ", root->keys[i]);
    }
    if(root->leaf == 0)
        display(root->child[i]);
}

//DELETE LEAF NODE OPERATION
void deleteLeaf(struct BTreeNode *root, int key)
{
    if(root == NULL)
        return;
    int i = 0;
    // Find the position of the key
    while(i < root->n && key > root->keys[i])
        i++;
    // Key found
    if(i < root->n && root->keys[i] == key)
    {
        if(root->leaf)
        {
            // Shift keys left
            for(int j = i; j < root->n - 1; j++)
                root->keys[j] = root->keys[j + 1];
            root->n--;
            printf("\n %d deleted successfully.\n", key);
        }
        else
        {
            printf("%d is in an internal node. Cannot delete using deleteLeaf().\n", key);
        }
        return;
    }
}

// If leaf and key not found
if(root->leaf)
{
    printf("%d not found.\n", key);
    return;
}
// Recur into the correct child
deleteLeaf(root->child[i], key);

```

```
}
```

```
//DISPLAY LEVEL OPERATION
```

```
void displayLevel(struct BTreeNode *root)
```

```
{
```

```
    if(root == NULL)
```

```
        return;
```

```
    struct BTreeNode *queue[100];
```

```
    int front = 0;
```

```
    int rear = 0;
```

```
    int level = 0;
```

```
    queue[rear++] = root;
```

```
    while(front < rear)
```

```
{
```

```
    int size = rear - front;
```

```
    printf("Level %d : ", level);
```

```
    while(size--)
```

```
{
```

```
    struct BTreeNode *temp = queue[front++];
```

```
    // Print keys of current node
```

```
    printf("[");
```

```
    for(int i = 0; i < temp->n; i++)
```

```
{
```

```
    printf("%d", temp->keys[i]);
```

```
    if(i != temp->n - 1)
```

```
        printf(" ");
```

```
}
```

```
    printf("] ");
```

```
    // Add children to queue
```

```
    if(temp->leaf == 0)
```

```
{
```

```
    for(int i = 0; i <= temp->n; i++)
```

```
{
```

```
    if(temp->child[i] != NULL)
```

```
        queue[rear++] = temp->child[i];
```

```
}
```

```
}
```

```
}
```

```
    printf("\n");
```

```
    level++;
```

```
}
```

```
}
```

```
//SEARCH OPERATION
```

```

struct BTreeNode *search(struct BTreeNode *root, int key)
{
    int i = 0;
    while(i < root->n && key > root->keys[i])
        i++;

    if(i < root->n && key == root->keys[i])
        return root;
    if(root->leaf)
        return NULL;
    return search(root->child[i], key);
}

int main()
{
    struct BTreeNode *root = createNode(1);
    int key;
    insert(&root, 10);
    insert(&root, 20);
    insert(&root, 5);
    insert(&root, 6);
    insert(&root, 12);
    insert(&root, 30);
    insert(&root, 7);
    insert(&root, 17);

    printf("B-Tree Level Order:\n");
    displayLevel(root);

    printf("B-Tree Traversal:\n");
    display(root);
    //DELETE ELEMENT
    printf("\n---DELETION OPERATION-----");
    printf("\n Before deletion:\n");
    printf("B-Tree Level Order:\n");
    displayLevel(root);
    printf("\nEnter key to delete: ");
    scanf("%d", &key);
    deleteLeaf(root, key);

    printf("\nAfter deletion:\n");
    printf("B-Tree Level Order:\n");
    displayLevel(root);

    //SEARCH ELEMENT
    printf("\n---SEARCH OPERATION-----");
    printf("\nEnter key to search: ");

```

```

scanf("%d", &key);
if(search(root, key) != NULL)
{
    printf("\n %d found.\n", key);
}
else
{
    printf("\n %d not found.\n", key);
}
return 0;
}

```

OUTPUT:

D:\>gcc BTreeNode.c -o BTreeNode.exe

D:\>BTreeNode.exe

B-Tree Level Order:

Level 0 : [10]

Level 1 : [5 6 7] [12 17 20 30]

B-Tree Traversal:

5 6 7 10 12 17 20 30

---DELETION OPERATION-----

Before deletion:

B-Tree Level Order:

Level 0 : [10]

Level 1 : [5 6 7] [12 17 20 30]

Enter key to delete: 12

12 deleted successfully.

After deletion:

B-Tree Level Order:

Level 0 : [10]

Level 1 : [5 6 7] [17 20 30]

---SEARCH OPERATION-----

Enter key to search: 10

10 found.

```
ca C:\Windows\system32\cmd.exe
D:\>BTreeNode.exe
B-Tree Level Order:
Level 0 : [10]
Level 1 : [5 6 7] [12 17 20 30]
B-Tree Traversal:
5 6 7 10 12 17 20 30
---DELETION OPERATION-----
Before deletion:
B-Tree Level Order:
Level 0 : [10]
Level 1 : [5 6 7] [12 17 20 30]
Enter key to delete: 12
12 deleted successfully.
After deletion:
B-Tree Level Order:
Level 0 : [10]
Level 1 : [5 6 7] [17 20 30]
---SEARCH OPERATION-----
Enter key to search: 10
10 found.
```

RESULT:

The B-Tree of order 5 was successfully implemented in C. The program successfully performed insertion, level-order display, inorder traversal, leaf node deletion, and search operations.